

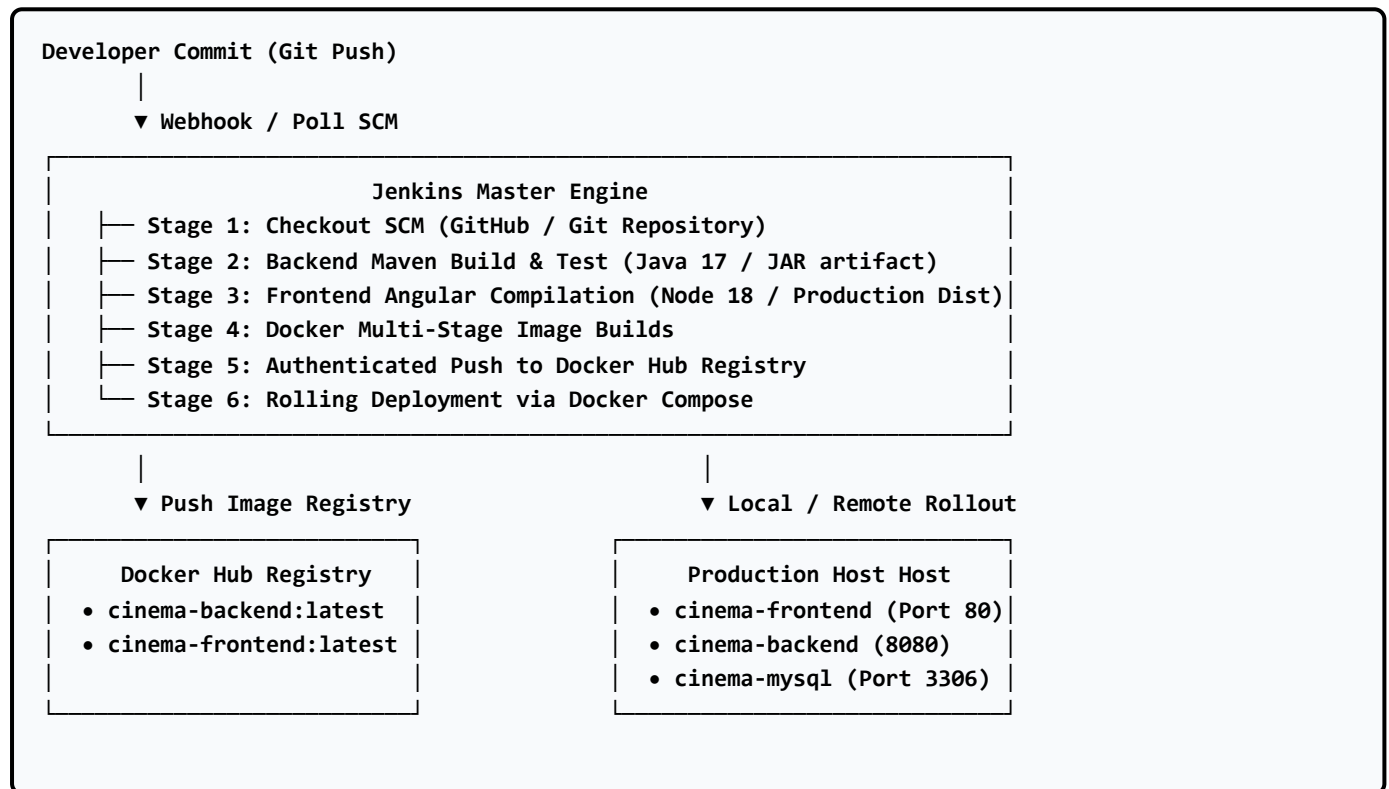
DEVOPS STANDARD OPERATING PROCEDURE (SOP)

Jenkins CI/CD Pipeline & Groovy Setup Manual

 **Project:** Hari's Theater Cinema Ticket Booking System | **Target:** Automated End-to-End Build, Push & Container Deployment

This manual provides exhaustive, step-by-step instructions for installing and provisioning a production Jenkins Continuous Integration and Continuous Deployment (CI/CD) engine. It standardizes automated testing, multi-stage Docker builds, Docker Hub registry publication, and Docker Compose runtime rollout via declarative Groovy code.

1. CI/CD Architecture Flow



2. Required Jenkins Plugins

Log in to Jenkins as Administrator, navigate to **Manage Jenkins > Plugins > Available Plugins**, search for and install the following packages without restart:

Plugin Name	Plugin ID	Core Responsibility
Pipeline	workflow-aggregator	Provides declarative Groovy DSL syntax orchestration.
Git Plugin	git	Automates repository polling, cloning, and branch management.
NodeJS Plugin	nodejs	Auto-provisions Node and npm runtimes on Jenkins build agents.
Docker Pipeline	docker-workflow	Enables native docker commands, authentication, and execution blocks.
Credentials Binding	credentials-binding	Allows secret credentials injection into build environments securely.

3. Global Tool Configuration (Exact Matching)

Navigate to **Manage Jenkins > Tools**. The names configured here **must match the names declared in the Groovy Jenkinsfile** exactly:

3.1 JDK Installation (Java 17)

- Click **Add JDK**.
- Name:** `JDK-17`
- Configuration:** Check *Install automatically* (Adoptium / Eclipse Temurin 17.x) OR provide local `JAVA_HOME` path (e.g. `C:\Program Files\Java\jdk-17` or `/usr/lib/jvm/java-17-openjdk`).

3.2 Maven Installation (Maven 3.9+)

- Click **Add Maven**.
- Name:** `Maven-3.9`
- Configuration:** Check *Install automatically* and select latest Apache Maven `3.9.x` version.

3.3 NodeJS Installation (NodeJS 18+)

- Click **Add NodeJS**.
- Name:** `NodeJS-18`
- Configuration:** Check *Install automatically* and select `NodeJS 18.x (LTS)`.

4. Docker Hub Secret Credentials Configuration

To safely authenticate and push container images to Docker Hub without hardcoding passwords in scripts:

1. Navigate to: **Manage Jenkins > Credentials > System > Global credentials (unrestricted)**.
2. Click **Add Credentials** in the top right.
3. Fill in the fields exactly as follows:

Configuration Field	Value to Enter
Kind	Username with password
Scope	Global (Jenkins, nodes, items, all child items)
Username	Your Docker Hub account username (e.g., hariadmin)
Password	Your Docker Hub Personal Access Token (PAT) or Account Password
ID	docker-hub-credentials (Must match Jenkinsfile variable)
Description	Docker Hub Registry Credentials for Cinema System

4. Click **Create**.

5. Step-by-Step Pipeline Job Creation

1. From the Jenkins Dashboard, select **New Item**.
2. Enter an item name: haris-theater-pipeline .
3. Select **Pipeline** as the project type and click **OK**.
4. In the configuration view:
 - Scroll down to the **Pipeline** section.
 - **Definition:** Change dropdown from *Pipeline script* to **Pipeline script from SCM**.
 - **SCM:** Select **Git**.
 - **Repository URL:** Enter your Git project URL (e.g., https://github.com/youruser/Movie-ticket-booking.git).
 - **Credentials:** Select your GitHub personal access token / SSH key if private.
 - **Branch Specifier:** */main (or */master).
 - **Script Path:** Jenkinsfile (Located in project root).
5. Click **Save**.
6. Click **Build Now** on the left sidebar to execute your first pipeline run.

6. Full Production Declarative Jenkinsfile (Groovy)

Place this exact code in `H:\PROJECT\Movie-ticket-booking\Jenkinsfile` :

```
pipeline {
    agent any

    environment {
        // Registry Credentials and Image Nomenclature
        DOCKER_HUB_CRED_ID = 'docker-hub-credentials'
        DOCKER_USER        = 'yourdockerhubusername' // Update with your Docker Hub ID
        BACKEND_IMAGE_NAME = "${DOCKER_USER}/cinema-backend"
        FRONTEND_IMAGE_NAME = "${DOCKER_USER}/cinema-frontend"
        IMAGE_TAG          = "${BUILD_NUMBER}"
    }

    tools {
        jdk 'JDK-17'
        maven 'Maven-3.9'
        nodejs 'NodeJS-18'
    }

    stages {
        stage('1. Checkout Source Code') {
            steps {
                echo 'Checking out code from version control...'
                checkout scm
            }
        }

        stage('2. Backend Build & Test') {
            steps {
                echo 'Compiling Spring Boot backend and producing runnable JAR...'
                dir('backend') {
                    script {
                        if (isUnix()) {
                            sh 'mvn clean package -DskipTests'
                        } else {
                            bat 'mvn clean package -DskipTests'
                        }
                    }
                }
            }
        }

        stage('3. Frontend Build') {
            steps {
                echo 'Installing Angular dependencies and generating production bundles...'
                dir('frontend') {
                    script {
                        if (isUnix()) {
                            sh 'npm ci'
                            sh 'npm run build -- --configuration production'
                        } else {
                            bat 'npm ci'
                            bat 'npm run build -- --configuration production'
                        }
                    }
                }
            }
        }
    }
}
```

```

stage('4. Build Docker Images') {
    steps {
        echo 'Building multi-tier Docker containers...'
        script {
            if (isUnix()) {
                sh "docker build -t ${BACKEND_IMAGE_NAME}:${IMAGE_TAG} -t ${BACKEND_IMAGE_NAME}:latest ./backend"
                sh "docker build -t ${FRONTEND_IMAGE_NAME}:${IMAGE_TAG} -t ${FRONTEND_IMAGE_NAME}:latest ./frontend"
            } else {
                bat "docker build -t ${BACKEND_IMAGE_NAME}:${IMAGE_TAG} -t ${BACKEND_IMAGE_NAME}:latest ./backend"
                bat "docker build -t ${FRONTEND_IMAGE_NAME}:${IMAGE_TAG} -t ${FRONTEND_IMAGE_NAME}:latest ./frontend"
            }
        }
    }
}

stage('5. Push to Docker Hub') {
    steps {
        echo 'Authenticating with Docker Hub and uploading versioned tags...'
        withCredentials([usernamePassword(credentialsId: "${DOCKER_HUB_CRED_ID}", usernameVariable: 'DH_USER', passwordVariable: 'DH_PASS')]) {
            script {
                if (isUnix()) {
                    sh "echo \${DH_PASS} | docker login -u \${DH_USER} --password-stdin"
                    sh "docker push ${BACKEND_IMAGE_NAME}:${IMAGE_TAG}"
                    sh "docker push ${BACKEND_IMAGE_NAME}:latest"
                    sh "docker push ${FRONTEND_IMAGE_NAME}:${IMAGE_TAG}"
                    sh "docker push ${FRONTEND_IMAGE_NAME}:latest"
                    sh "docker logout"
                } else {
                    bat "echo %DH_PASS% | docker login -u %DH_USER% --password-stdin"
                    bat "docker push ${BACKEND_IMAGE_NAME}:${IMAGE_TAG}"
                    bat "docker push ${BACKEND_IMAGE_NAME}:latest"
                    bat "docker push ${FRONTEND_IMAGE_NAME}:${IMAGE_TAG}"
                    bat "docker push ${FRONTEND_IMAGE_NAME}:latest"
                    bat "docker logout"
                }
            }
        }
    }
}

stage('6. Deploy with Docker Compose') {
    steps {
        echo 'Rolling deployment: refreshing services on target node...'
        script {
            if (isUnix()) {
                sh 'docker compose down'
                sh 'docker compose up -d --build'
            } else {
                bat 'docker compose down'
                bat 'docker compose up -d --build'
            }
        }
    }
}

```

```
    }  
  }  
}  
  
post {  
  success {  
    echo "=====  
    echo "Pipeline Passed! Hari's Theater is LIVE on http://localhost"  
    echo "=====  
  }  
  failure {  
    echo "=====  
    echo "Pipeline Failed! Inspect Stage View logs for diagnosis."  
    echo "=====  
  }  
  always {  
    echo 'Pruning dangling temporary images to optimize disk volume...'  
    script {  
      if (isUnix()) {  
        sh 'docker image prune -f'  
      } else {  
        bat 'docker image prune -f'  
      }  
    }  
  }  
}  
}
```

7. CI/CD Troubleshooting & Diagnostic Runbook

Encountered Error	Probable Root Cause	Remediation Step
Tool [JDK-17] was not found	Mismatched naming in Global Tool Configuration.	Navigate to Manage Jenkins > Tools and verify exact spelling of [JDK-17].
docker: permission denied	The [jenkins] system user is not a member of the host [docker] security group.	Run on host: <code>sudo usermod -aG docker jenkins</code> and restart Jenkins service (<code>sudo systemctl restart jenkins</code>).
denied: requested access to the resource is denied	Invalid Docker Hub credentials or mismatched image repository prefix.	Ensure [DOCKER_USER] matches the account that owns the Docker Hub namespace, and re-check [docker-hub-credentials].
npm: command not found	NodeJS tool not configured or missing from PATH on agent.	Configure [NodeJS-18] in Global Tool Configuration and verify the plugin is active.

8. Production Verification Checklist

- Pipeline Stage View:** Verify all 6 stages display Green checkmarks in the Jenkins Web UI.
- Docker Hub Registry:** Visit [hub.docker.com] and confirm repositories [cinema-backend] and [cinema-frontend] received updated build tags.
- Runtime Ingress:** Access [http://localhost] to confirm the production Angular application displays all 9 movie posters without 404 errors.
- Backend Health:** Access [http://localhost:8080/api/movies] to verify JSON catalog streaming directly from the MySQL database container.